

Сильная база в computer science, алгоритмах и исследовательском решении задач; практический опыт C++ и Python, LLVM и compiler/runtime-разработки. Работаю в цикле «гипотеза → эксперимент → проверка»: строю точные оракулы, randomized/metamorphic tests, воспроизводимые harnesses и benchmarks, анализирую контрпримеры и производительность. Работаю с JIT-компиляцией, векторизацией, x86-64 assembly и анализом генерируемого машинного и дизассемблированного кода

Алгоритмы и соревнования

Призёр «Высшей пробы»; абсолютный победитель конкурса «Юниор» НИЯУ МИФИ в 2025 и 2026 годах; Главная премия Балтийского научно-инженерного конкурса

Гипотеза → эксперимент

Точные оракулы, randomized/metamorphic testing, adversarial counterexamples, воспроизводимые harnesses и benchmarks

C++ / low-level performance

C++23, LLVM, JIT, векторизация, x86-64 assembly, анализ generated/disassembled machine code и benchmarks

ОПЫТ		КОМПЕТЕНЦИИ
1 июля — 31 августа 2026	МЦСТ — стажёр по разработке компиляторов - Реализовал LLVM-проход loop-invariant code motion на C++; анализировал инвариантность, side effects, обращения к памяти, speculative safety и структуру циклов - Построил Python-harness, который сравнивает поведение до и после оптимизации и отдельно проверяет ожидаемые случаи transformation / no-transformation	Programming C++23, Python, C17, C#, Rust Алгоритмы и CS структуры данных, алгоритмы графов, program analysis, CFG/SSA, compiler optimizations Low-level & performance LLVM, JIT/CIL, x86-64 assembly, vectorization, compiler optimizations, generated/disassembled machine-code analysis, benchmarking Эксперименты и tooling формулировка и проверка гипотез, exact oracles, differential/metamorphic/randomized testing, Python-harnesses, Linux, Git, CMake
2026	PS-form Memory Dependence Analyzer - Занял Консервативный анализ проверен независимым exact oracle на малых областях, randomized/metamorphic cases и воспроизводимыми контрпримерами - Построил точный оракул для малых областей, metamorphic/randomized проверки и воспроизводимые WA/TL-контрпримеры	
2024 — сейчас	UniversalToolchain — создатель и основной разработчик - Самостоятельно спроектировал и развиваю сложную языковую систему с промежуточными представлениями, SSA/оптимизациями и interpreter/CIL backend-ами - Использую benchmarking, JIT/CIL execution и анализ сгенерированного и дизассемблированного кода для проверки производительности и поведения реализаций	ОБРАЗОВАНИЕ НИУ ВШЭ — Программная инженерия, 2026–2030 ДОСТИЖЕНИЯ Диплом I степени и Главная премия «Совершенство как надежда» за UniversalToolchain. Москва / удалённо

ПРОЕКТЫ

PS-form Memory Dependence Analyzer
Консервативный результат yes/no/maybe с независимым exact oracle на малых областях, randomized/metamorphic проверками и воспроизводимыми контрпримерами.

UniversalToolchain
LanguageRuntime проверяет exact planned package/component graph и создаёт выбранные компоненты без повторного планирования; контракт защищён determinism/exact-binding checks, ownership guards и interpreter/CIL parity.

x86-64 Codegen & Register Allocation Lab
Reference interpreter и differential execution проверяют семантическую эквивалентность сгенерированного кода; allocator contract валидируется отдельно.